Lab: HTTP/2 Protocol Analysis - Encrypted Performance & Packet Forensics

Author: Supavit Pitaknarongphorn
Course: Network Forensics
Objective: Analyze HTTP/2 transactions between Linux client and server with TLS encryption

Introduction

TLS, or Transport Layer Security, is a fundamental cryptographic protocol designed to provide secure communication over a computer network. It is the modern successor to SSL (Secure Sockets Layer) and ensures privacy and data integrity between two communicating applications, such as a web browser and a server. By encrypting the data sent over the internet, TLS prevents eavesdropping, tampering, and message forgery. It's the technology behind the padlock icon and "https://" in your web browser, safeguarding sensitive information like passwords, credit card details, and personal messages during online transactions and browsing.

HTTP/2 represents a major evolution in web protocol design, introducing multiplexing, header compression, and server push capabilities while maintaining compatibility with HTTP/1.1 semantics. Unlike its predecessor, HTTP/2 operates over a single TCP connection, reducing latency and improving page load times. However, when combined with TLS encryption (as required by most modern browsers), analyzing these transactions requires specialized tools and techniques. In this lab, you'll configure an HTTP/2 server, capture encrypted transactions, and use network forensic tools to analyze the protocol's behavior despite the encryption layer.

Modern web protocols like HTTP/2 present unique challenges for network forensic analysts. While HTTP/1.1 operated as plain text over port 80, HTTP/2 is typically deployed exclusively over TLS-encrypted connections (port 443). This encryption, while essential for security and privacy, creates a "black box" effect where traditional packet inspection tools cannot directly observe application-layer protocol behavior. Forensic analysts must therefore employ specialized techniques to analyze these encrypted transactions, including SSL/TLS key logging, protocol fingerprinting through metadata

analysis, and examining protocol negotiation patterns in the unencrypted portions of TLS handshakes.

Tools and Applications

NGINX is a high-performance, open-source web server and reverse proxy. It's known for efficiently handling many simultaneous connections with minimal resources, using an event-driven architecture. It's widely used for serving web content, load balancing, caching, and improving application security and speed.

SSL/TLS certificates are digital credentials that enable secure, encrypted communication over the internet between a web server and a browser. They work by implementing the SSL (Secure Sockets Layer) and TLS (Transport Layer Security) protocols to create an encrypted link, ensuring that all data passed between the server and client, such as credit card information, login details, or personal messages, remains private and integral.

TShark is the command-line version of the popular network protocol analyzer Wireshark. It captures and analyzes network traffic directly from a terminal or script, using the same powerful dissection and filtering engines as its graphical counterpart. This makes TShark ideal for automation, remote server analysis, and situations where a graphical interface is unavailable or inefficient. Users can capture live packets, read from saved files, apply display filters, and output analysis in various formats, providing a versatile tool for network troubleshooting, security monitoring, and data forensics from the command line.

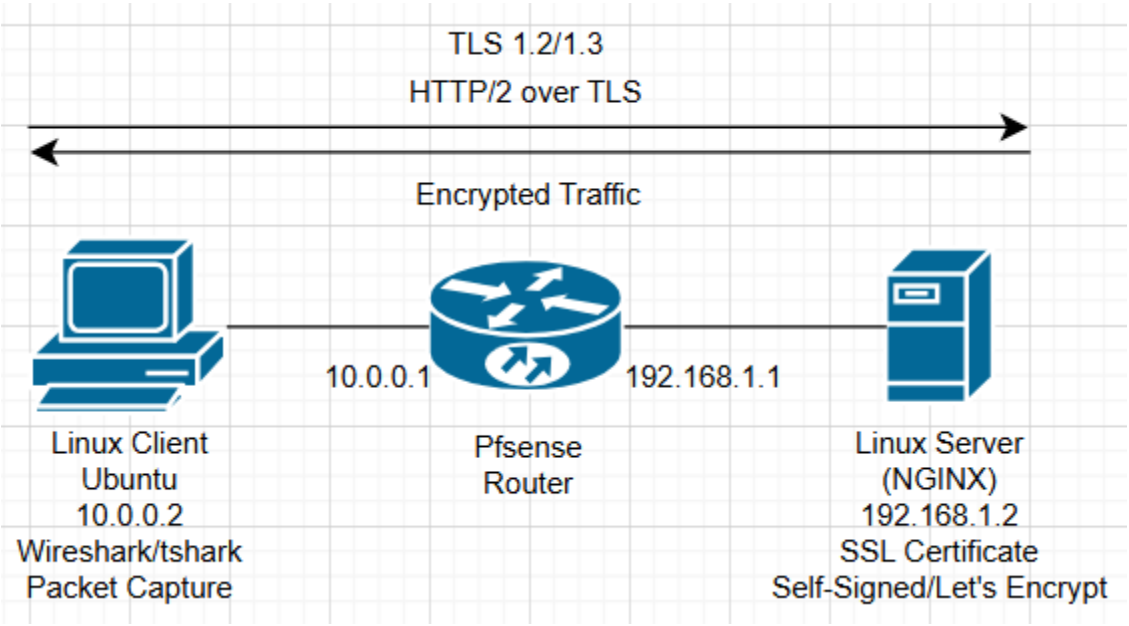
Wireshark is a free and open-source network protocol analyzer, often described as the world's foremost tool for examining network traffic in real time. It allows you to capture data packets as they travel across a network, dissecting and displaying them in a detailed, human-readable format. Network administrators, security professionals, and developers use Wireshark to troubleshoot problems, analyze protocol behavior, investigate security incidents, and debug application communication. By providing a deep, granular view of every byte transmitted, it serves as an essential instrument for understanding and maintaining modern networks.

Learning Objectives

- Configure NGINX with HTTP/2 and TLS encryption
- Capture HTTP/2 traffic using packet capture tools

- Analyze protocol characteristics despite TLS encryption
- Identify HTTP/2 specific features in network traces
- Understand the forensic challenges of encrypted modern protocols

Network Diagram



WAN: 10.0.0.0/24
LAN: 192.168.1.0/24

Credentials

Linux Client

- **Username:** student
- **Password:** tartans

Linux Server

- **Username:** student
- **Password:** tartans

Phase 1: HTTP/2 Server Configuration

Step 1.1: Install NGINX on Server VM

Go to **Linux Server**. Start the Nginx web server service.

sudo systemctl start nginx

Configure Nginx to start automatically on system boot

sudo systemctl enable nginx

```
student@ubuntu:~$ sudo systemctl start nginx
student@ubuntu:~$ sudo systemctl enable nginx
Synchronizing state of nginx.service with SysV service script with /
usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable nginx
student@ubuntu:~$
```

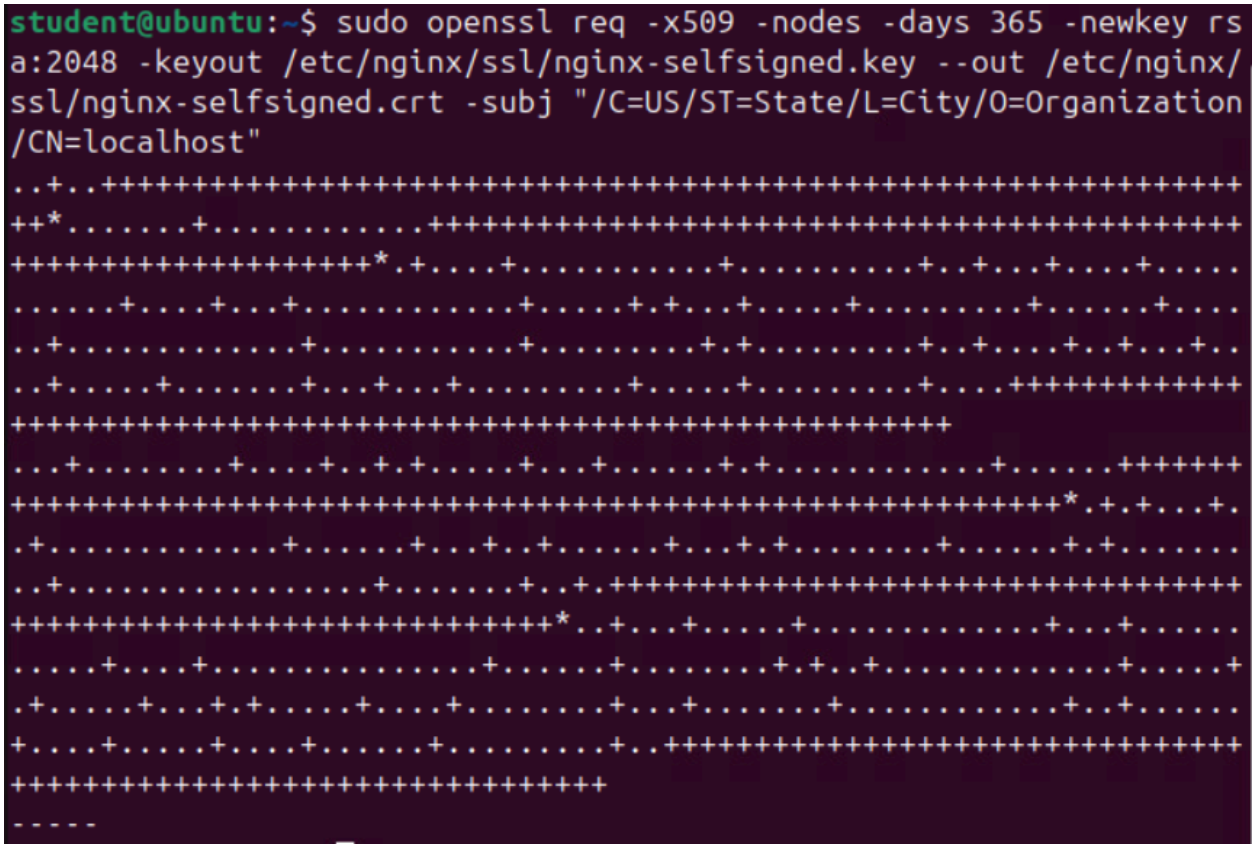
Step 1.2: Generate SSL/TLS Certificates

Create a directory for SSL/TLS certificates used by the Nginx web server:

sudo mkdir -p /etc/nginx/ssl

Generate a self-signed SSL certificate and private key valid for 365 days for localhost:

sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /etc/nginx/ssl/nginx-selfsigned.key -out /etc/nginx/ssl/nginx-selfsigned.crt -subj "/C=US/ST=State/L=City/O=Organization/CN=localhost"



Step 1.3: Configure NGINX for HTTP/2 with TLS

Remove the existing default Nginx site configuration file so that we can create our own:

sudo rm /etc/nginx/sites-available/default

Create the default Nginx site configuration file:

sudo nano /etc/nginx/sites-available/default

Replace the file content with the content below (in red) to set up an HTTPS server for localhost using the self-signed SSL certificate you created. This is what the content does:

1. **HTTPS Listening:** Listens on port 443 for both IPv4 and IPv6 with SSL and HTTP/2
2. **SSL Configuration:** References your self-signed certificate and key
3. **Security Settings:** Uses TLS 1.2/1.3 with secure cipher suites
4. **Web Root:** Serves files from /var/www/html

5. **Routing:** Serves static files or returns 404 for missing files

```
server {
    listen 443 ssl http2;
    listen [::]:443 ssl http2;

    server_name localhost;

    ssl_certificate /etc/nginx/ssl/nginx-selfsigned.crt;
    ssl_certificate_key /etc/nginx/ssl/nginx-selfsigned.key;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    root /var/www/html;
    index index.html index.htm;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

Press **Ctrl-S** to save, and **Ctrl-X** to exit.



Step 1.4: Test and Enable Configuration

Test the Nginx configuration for syntax errors before reloading the service.

sudo nginx -t

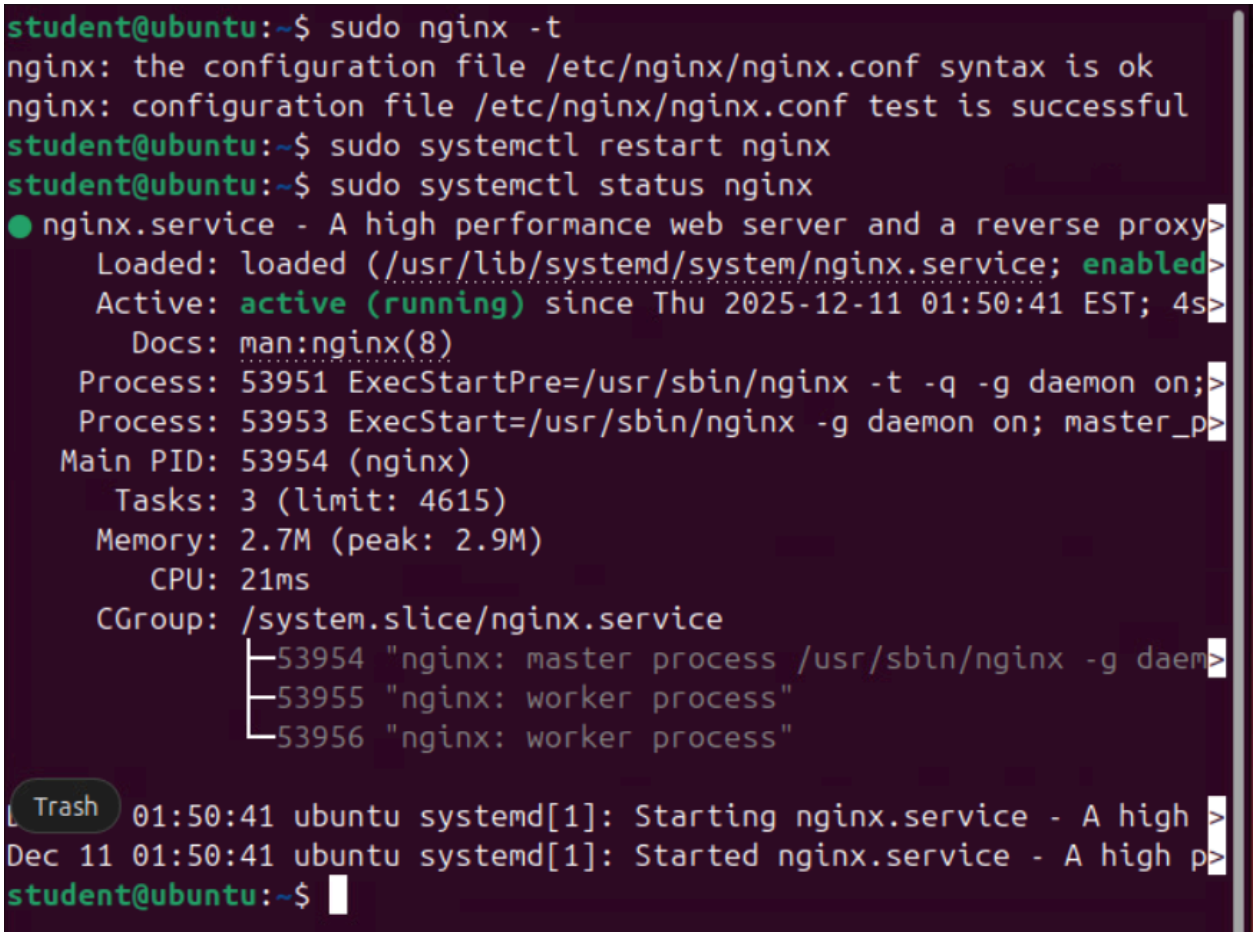
Restart the Nginx web server to apply configuration changes

sudo systemctl restart nginx

Check the current status of the Nginx web server service

sudo systemctl status nginx

Press **q** to exit the display



Step 1.5: Prepare Test Webpage

Open the default Nginx web root's index.html file

sudo nano /var/www/html/index.html

Create a simple HTML test page for verifying HTTP/2 functionality with two image references.

```
<!DOCTYPE html>
<html>
<head>
  <title>HTTP/2 Test Page</title>
</head>
<body>
  <h1>HTTP/2 Lab Test Page</h1>
  <p>Testing HTTP/2 protocol analysis.</p>
```

```
    <img src="image1.jpg" alt="Test Image 1">
    <img src="image2.jpg" alt="Test Image 2">
</body>
</html>
```

```
 GNU nano 7.2              /var/www/html/index.html
<!DOCTYPE html>
<html>
<head>
    <title>HTTP/2 Test Page</title>
</head>
<body>
    <h1>HTTP/2 Lab Test Page</h1>
    <p>Testing HTTP/2 protocol analysis.</p>
    <img src="image1.jpg" alt="Test Image 1">
    <img src="image2.jpg" alt="Test Image 2">
</body>
</html>
```

Press **Ctrl-S** to save, and **Ctrl-X** to exit.

Step 1.6: Create Test Resources

Create two colored test images for testing

```
sudo convert -size 800x600 xc:skyblue /var/www/html/image1.jpg
sudo convert -size 800x600 xc:forestgreen /var/www/html/image2.jpg
```

Allow firewall to accept tcp connetions

```
sudo ufw allow 443/tcp
```

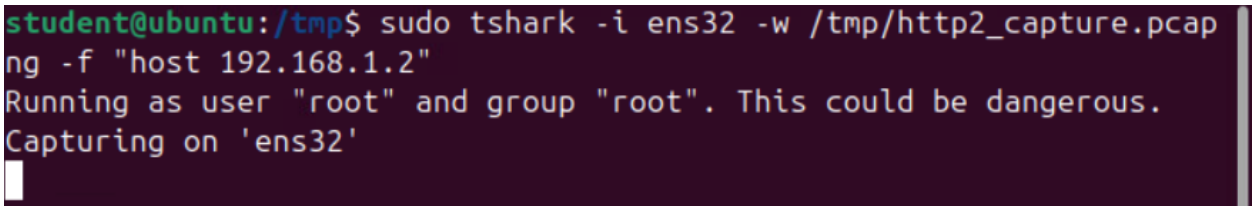
```
student@ubuntu:~$ sudo convert -size 800x600 xc:skyblue /var/www/html/image1.jpg
student@ubuntu:~$ sudo convert -size 800x600 xc:forestgreen /var/www/html/image2.jpg
student@ubuntu:~$ sudo ufw allow 443/tcp
Skipping adding existing rule
Skipping adding existing rule (v6)
student@ubuntu:~$
```

Phase 2: Client Setup And Traffic Capture

Step 2.1: Start Packet Capture on Client

Go to **Linux Client.** Start capture on the ens32 interface with the filter "host 192.168.1.2" to find packets associated with the Linux server

sudo tshark -i ens32 -w /tmp/http2_capture.pcapng -f "host 192.168.1.2"



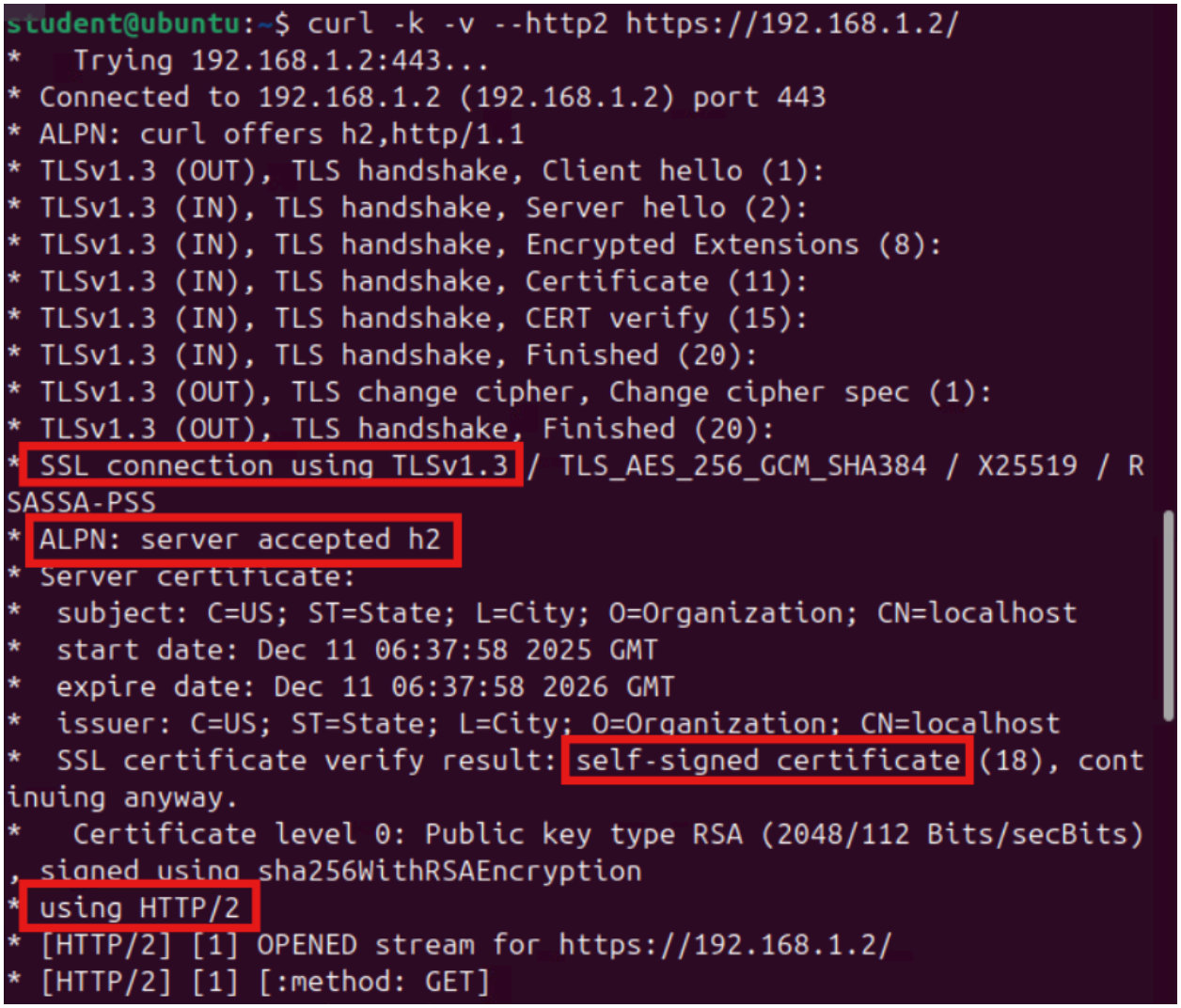
Create a new terminal while tshark is capturing.

Step 2.2: Generate HTTP/2 Traffic

Test with curl. It should show the following to verify that the setup is working correctly:

1. **HTTP/2 Successfully Negotiated:** ALPN: server accepted h2
2. **TLS 1.3 Working:** SSL connection using TLSv1.3
3. **Self-signed Certificate Accepted:** Certificate verification bypassed with -k flag
4. **NGINX Server Responding:** server: nginx/1.24.0 (Ubuntu)
5. **HTTP/2 Headers:** Shows HTTP/2 specific headers like :method, :scheme, :authority

curl -k -v --http2 https://192.168.1.2/



Test with browser-like request (multiple parallel requests). The garbage output is usual when performing parallel requests

```
for i in {1..5}; do
   curl -k --http2 https://192.168.1.2/image$i.jpg -o /dev/null &
done
wait
```

Press **Ctrl-C** to return to command prompt.

```
student@ubuntu:~$ for i in {1..5}; do
>     curl -k --http2 https://192.168.1.2/image$i.jpg -o /dev/null &
> done
[2] 37979
[3] 37980
[4] 37981
[5] 37982
[6] 37983
student@ubuntu:~$   % Total    % Received % Xferd  Average Speed   T
ime    Time     Time  Current
                                 Dload  Upload   Total   Spent    Le
ft  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:-
-:--     0wait
  % To  % Total    % Received % Xferd   % tal    % ReceTiovtale  d
 % % Xfer Rdecei ve Ad v%  AvXfeerage Speeed   Time    Time     Time
  Current
 rage Speedrd   Ave rage  SpeeTd   iTmieme                    TT ii
 m m ee             %    TTi Tom ite ma el  C   u    rC  rue%  rn  D
rReet
 lnc oate d
i   v e d    U  p%l  o a  dX f   e   r  d         To A t ve   r  aa
l g   S            Dloepen  a d  U St     ploape Lef  d   edt  S  To
ta  peed  l
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:-
-:--     0  Spe    DlToime    Tnimte        L eTfitm e  S pCeuerdr
Show Apps
```

Step 2.3: Stop Packet Capture

Go to the **terminal where tshark is capturing**. Press **Ctrl-C** to stop capturing.

```
student@ubuntu:/tmp$ sudo tshark -i ens32 -w /tmp/http2_capture.pcap
ng -f "host 192.168.1.2"
Running as user "root" and group "root". This could be dangerous.
Capturing on 'ens32'
147 ^C
student@ubuntu:/tmp$
```

Allow permissions for http2_capture.pcapng file

sudo chmod 777 /tmp/http2_capture.pcapng

Phase 3: Encrypted Packet Analysis

Step 3.1: Basic Capture Statistics

Count packets

tshark -r /tmp/http2_capture.pcapng | wc -l

List protocols observed.

tshark -r /tmp/http2_capture.pcapng -qz io,phs

HTTP/2 is not visible because it operates within TLS-encrypted Application Data records. The encryption renders HTTP/2 frames invisible to standard packet analysis tools, appearing only as opaque TLS traffic.

Show TLS handshake details.

tshark -r /tmp/http2_capture.pcapng -Y "tls.handshake.type == 1"

```
student@ubuntu:~$ sudo chmod 777 /tmp/http2_capture.pcapng
student@ubuntu:~$ tshark -r /tmp/http2_capture.pcapng | wc -l
176
student@ubuntu:~$ tshark -r /tmp/http2_capture.pcapng -qz io,phs

===================================================================
Protocol Hierarchy Statistics
Filter:

eth                                      frames:176 bytes:42736
  ip                                     frames:176 bytes:42736
    tcp                                  frames:170 bytes:42148
      tls                                frames:73 bytes:29754
        tcp.segments                     frames:8 bytes:3917
    icmp                                 frames:6 bytes:588
===================================================================
```

```
student@ubuntu:~$ tshark -r /tmp/http2_capture.pcapng -Y "tls.handsh
ake.type == 1"
   28 352.167083964     10.0.0.2 → 192.168.1.2  TLSv1 583 Client Hel
lo
   50 705.336919038     10.0.0.2 → 192.168.1.2  TLSv1 583 Client Hel
lo
   68 705.347224998     10.0.0.2 → 192.168.1.2  TLSv1 583 Client Hel
lo
   69 705.347236509     10.0.0.2 → 192.168.1.2  TLSv1 583 Client Hel
lo
   77 705.351415658     10.0.0.2 → 192.168.1.2  TLSv1 583 Client Hel
lo
   90 705.352715094     10.0.0.2 → 192.168.1.2  TLSv1 583 Client Hel
lo
```

Step 3.2: Identify HTTP/2 Frames

Extract HTTP/2 frame types

tshark -r /tmp/http2_capture.pcapng -Y "http2" -T fields -e frame.number -e ip.src -e ip.dst -e http2.type | head -20

HTTP/2 frames are not visible because they're encrypted within TLS Application Data records.

```
student@ubuntu:~$ tshark -r /tmp/http2_capture.pcapng -Y "http2" -T
fields -e frame.number -e ip.src -e ip.dst -e http2.type | head -20
student@ubuntu:~$
```

Step 3.3: Wireshark Analysis

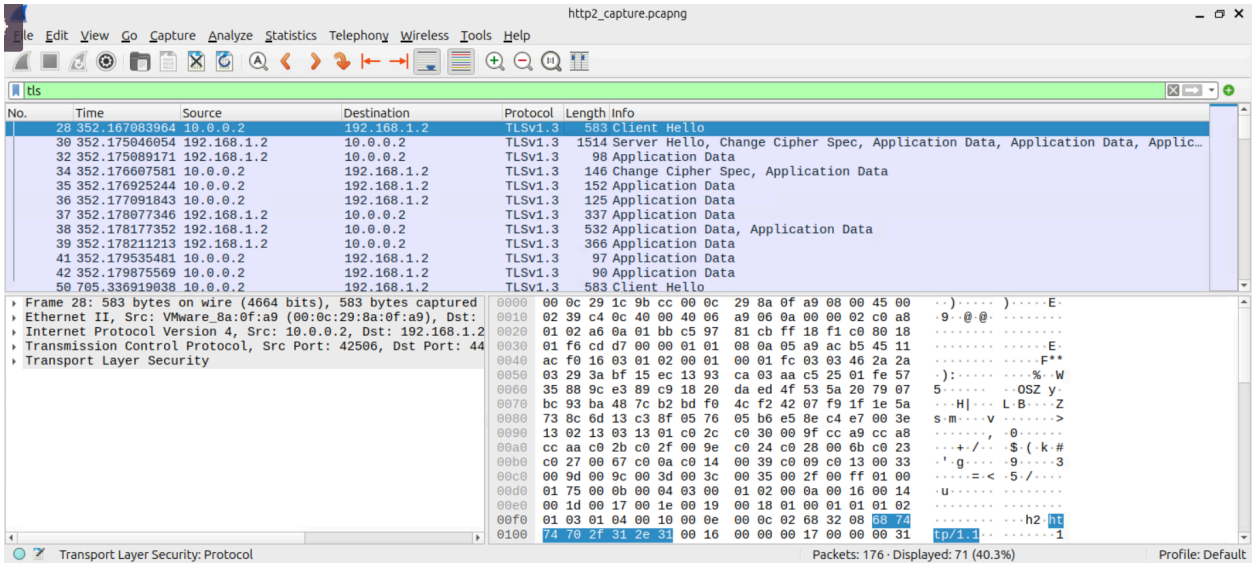
Load Capture in Wireshark

wireshark /tmp/http2_capture.pcapng &

Apply filter as “tls” (without quotation marks). Take a look at the packets. You will see the following:

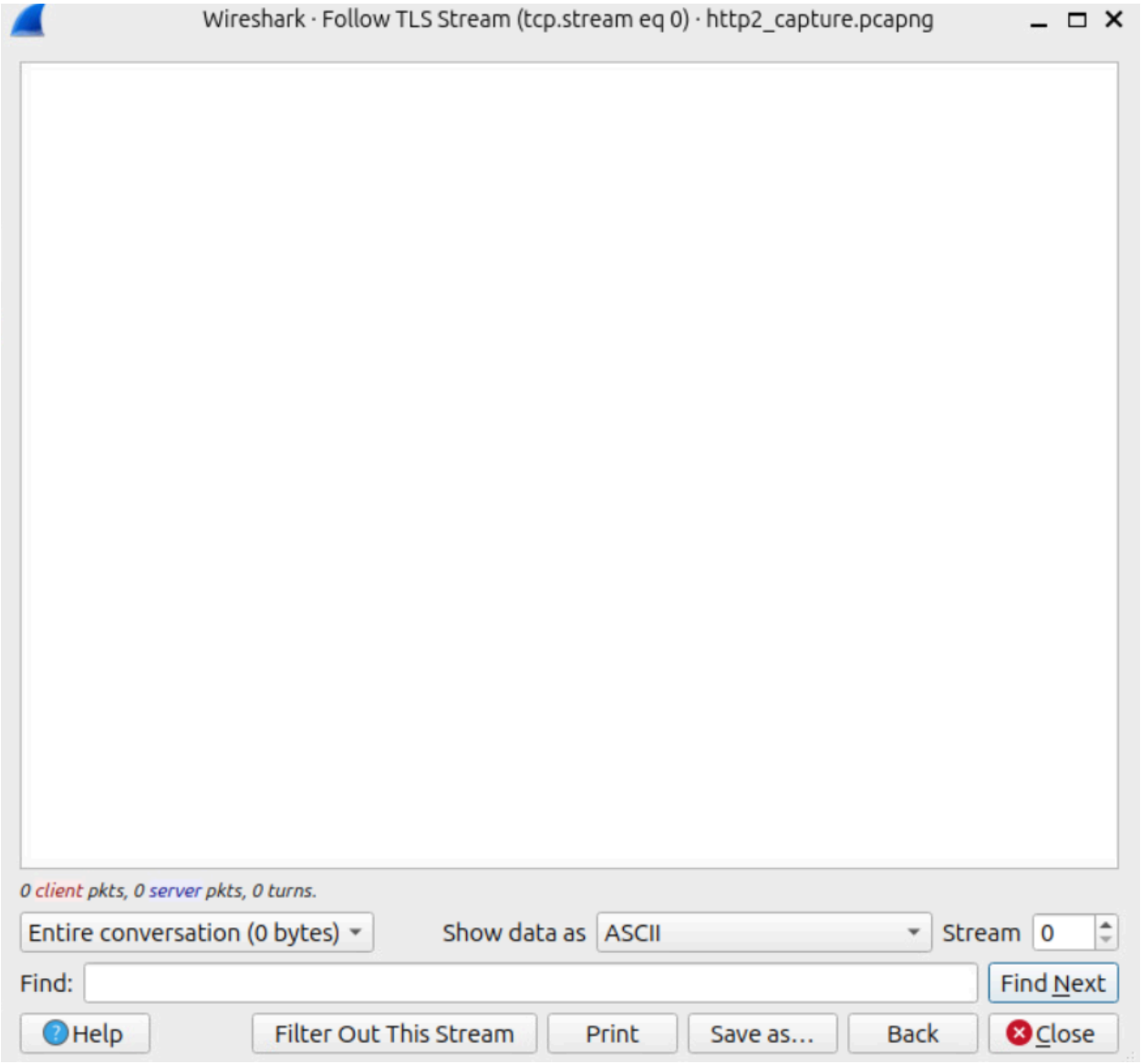
1. Client Hello
2. Server Hello
3. Certificate Exchange

4. Key Exchange
5. Application Data



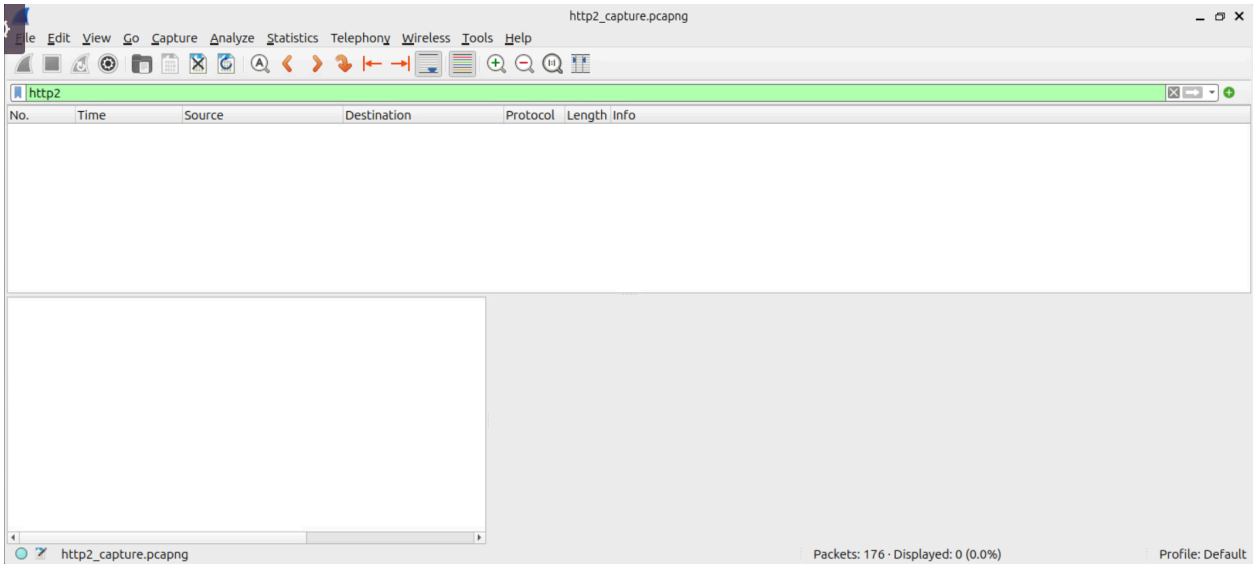
Step 3.4: Follow TLS Stream

Right-click on TLS frame with **Application Data. Follow > TLS Stream.** You will **see nothing** due to encryption.



Step 3.5: Analyze HTTP/2 in Wireshark

Apply filter as “http2” (without quotation marks). You can also see that it is **not showing anything** due to encryption.



Phase 4: Decryption

In real-world scenarios, extracting SSL/TLS keys is extremely difficult and often impossible. Production environments typically prohibit SSL key logging because the keys themselves become high-value targets that could compromise all encrypted communications. The methods shown in this lab are primarily educational and rarely applicable outside controlled testing environments or with explicit legal authorization.

Step 4.1: Extract SSL Keys for Decryption

Go to the **Linux Server**. Configure NGINX to log SSL keys

sudo nano /etc/nginx/nginx.conf

Under HTTP > SSL Settings Section, add the following:

ssl_session_tickets off;

The command disables TLS session tickets, which are an optimization that allows clients to resume sessions without a full handshake. By turning this off, NGINX forces full TLS handshakes that include the generation of new session keys. This enables SSL key logging to capture the necessary encryption materials for decrypting traffic during forensic analysis.



Press **Ctrl-S** to save, and **Ctrl-X** to exit. Restart nginx.

```
sudo systemctl restart nginx
```

Step 4.2: Generate new traffic with key log file

Go to **Linux Client.** Start new capture

```
sudo tshark -i ens32 -w /tmp/http2_decrypted.pcapng -f "host 192.168.1.2"
```

Go to a **new terminal** while tshark is capturing. Store key log file path.

```
export SSLKEYLOGFILE=/tmp/sslkeys.log
```

The command sets an environment variable that instructs applications (like curl) to write TLS session keys to a specified file. This key log file contains the cryptographic material needed to decrypt captured TLS traffic later. By generating this file during traffic generation, we enable Wireshark and tshark to decrypt and analyze the HTTP/2 frames that were previously hidden within encrypted TLS Application Data.

Generate traffic again

```
curl -k --http2 https://192.168.1.2 > /dev/null
```

Generate parallel requests again

```
for i in {1..5}; do
    curl -k --http2 https://192.168.1.2/image$i.jpg -o /dev/null &
done
```

wait

Press **Ctrl-C** to return to command prompt. Go to the **terminal where tshark is capturing**. Press **Ctrl-C** to stop capturing. Allow permissions for http2_decrypted.pcapng file.

sudo chmod 777 /tmp/http2_decrypted.pcapng

Phase 5: Tshark Packet Analysis

Step 5.1: Basic Capture Statistics

Count packets

tshark -r /tmp/http2_decrypted.pcapng -o tls.keylog_file:/tmp/sslkeys.log | wc -l

The -o tls.keylog_file:/tmp/sslkeys.log option tells tshark/Wireshark where to find the SSL/TLS key log file you created. This provides the decryption keys needed to interpret the encrypted TLS Application Data packets. With this option, previously invisible HTTP/2 frames become visible for analysis because the tools can now decrypt the traffic.

List protocols observed

tshark -r /tmp/http2_decrypted.pcapng -o tls.keylog_file:/tmp/sslkeys.log -qz io,phs

In the protocol hierarchy with key log decryption, HTTP/2 now appears as a visible protocol layer because tshark can decrypt the TLS Application Data records. The key log file provides the necessary encryption keys, allowing tshark to interpret the previously opaque binary data as HTTP/2 frames. This reveals the complete protocol stack including HTTP/2, demonstrating that decryption enables full protocol visibility for forensic analysis.

Show TLS handshake details

tshark -r /tmp/http2_decrypted.pcapng -o tls.keylog_file:/tmp/sslkeys.log -Y "tls.handshake.type == 1"

```
student@ubuntu:/tmp$ tshark -r /tmp/http2_decrypted.pcapng -o tls.ke
ylog_file:/tmp/sslkeys.log | wc -l
151
student@ubuntu:/tmp$ tshark -r /tmp/http2_decrypted.pcapng -o tls.ke
ylog_file:/tmp/sslkeys.log -qz io,phs

===================================================================
Protocol Hierarchy Statistics
Filter:

eth                                      frames:151 bytes:38486
  ip                                     frames:151 bytes:38486
    tcp                                  frames:151 bytes:38486
      tls                                frames:74 bytes:26068
        tcp.segments                     frames:6 bytes:588
        http2                            frames:30 bytes:6714
          http2                          frames:12 bytes:2313
            http2                        frames:12 bytes:2313
              http2                      frames:6 bytes:1401
          data-text-lines                frames:4 bytes:1143
          image-jfif                     frames:2 bytes:1881
            tcp.segments                 frames:2 bytes:1881
===================================================================
```

```
student@ubuntu:/tmp$ tshark -r /tmp/http2_decrypted.pcapng -o tls.ke
ylog_file:/tmp/sslkeys.log -Y "tls.handshake.type == 1"
    4 0.003119799     10.0.0.2 → 192.168.1.2  TLSv1 583 Client Hello
   30 5.861585601     10.0.0.2 → 192.168.1.2  TLSv1 583 Client Hello
   31 5.861947033     10.0.0.2 → 192.168.1.2  TLSv1 583 Client Hello
   43 5.866938195     10.0.0.2 → 192.168.1.2  TLSv1 583 Client Hello
   52 5.869786868     10.0.0.2 → 192.168.1.2  TLSv1 583 Client Hello
   79 5.873586500     10.0.0.2 → 192.168.1.2  TLSv1 583 Client Hello
```

Step 5.2: Identify HTTP/2 Frames

Extract HTTP/2 frame types

tshark -r http2_decrypted.pcapng -o tls.keylog_file:/tmp/sslkeys.log -Y "http2" -T fields -e frame.number -e ip.src -e ip.dst -e http2.type | head -20

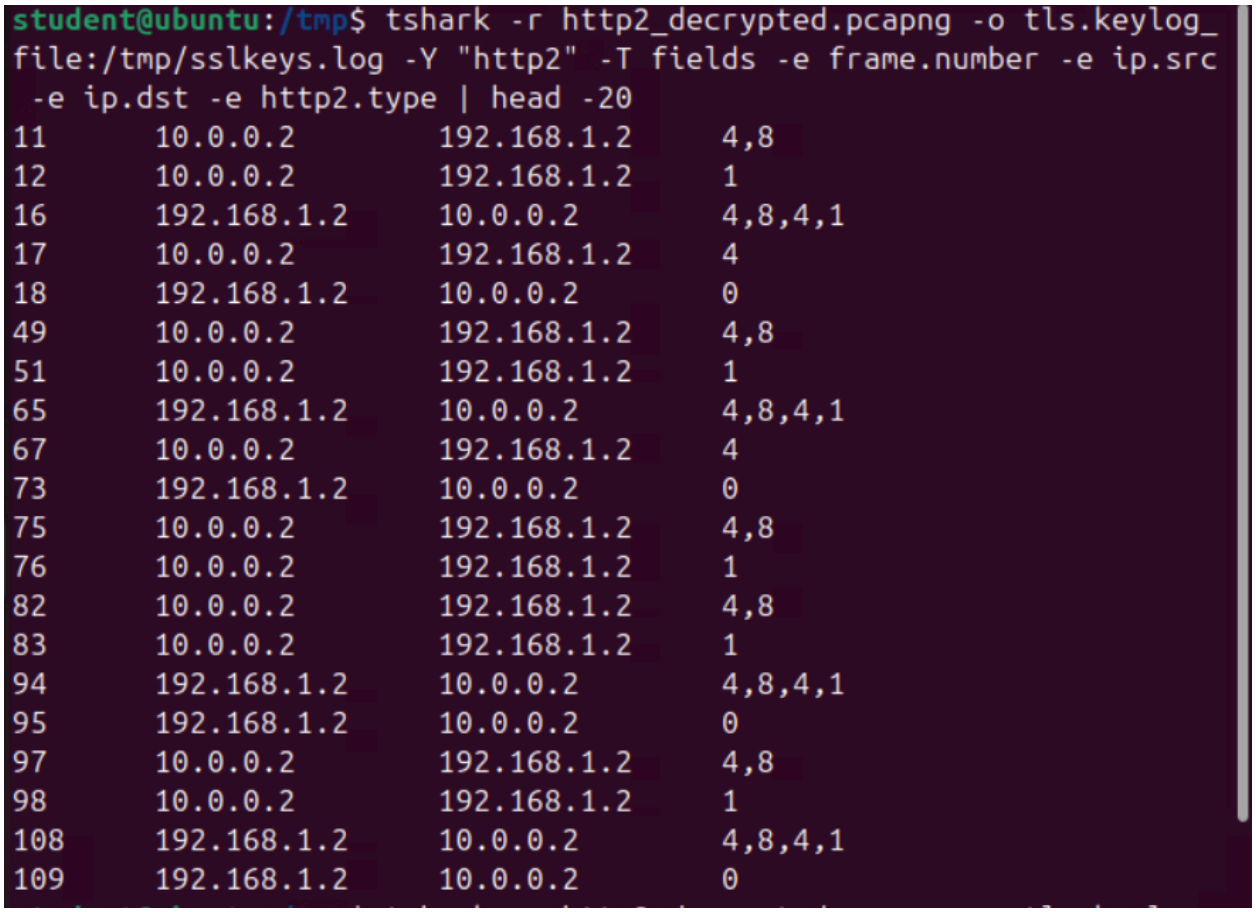
we now see HTTP/2 frame details because the key log file enables tshark to decrypt the TLS traffic. The output shows specific HTTP/2 frame types (HEADERS, DATA, SETTINGS, etc.) along with their source/destination IPs and frame numbers. This

successful decryption reveals HTTP/2's binary framing layer that was previously hidden within encrypted TLS Application Data packets.

Count different HTTP/2 frame types

tshark -r http2_decrypted.pcapng -o tls.keylog_file:/tmp/sslkeys.log -Y "http2" -T fields -e http2.type | sort | uniq -c | sort -rn

This command reveals the distribution of different HTTP/2 frame types within the decrypted traffic. By counting each frame type, we can see which HTTP/2 features are being utilized (like HEADERS for requests, DATA for content, SETTINGS for configuration). This analysis is only possible because decryption exposes the HTTP/2 protocol structure that was previously hidden within encrypted TLS packets.



```
student@ubuntu:/tmp$ tshark -r http2_decrypted.pcapng -o tls.keylog_
file:/tmp/sslkeys.log -Y "http2" -T fields -e http2.type | sort | un
iq -c | sort -rn
      6 4,8,4,1
      6 4,8
      6 4
      6 1
      6 0
```

Step 5.3: Analyze Stream Multiplexing

Show HTTP/2 streams

tshark -r http2_decrypted.pcapng -o tls.keylog_file:/tmp/sslkeys.log -Y "http2" -T fields -e http2.streamid -e http2.type -e http2.length | sort -n | head -30

This command reveals HTTP/2 stream multiplexing by showing different stream IDs (1, 3, 5, etc.) within the same TCP connection. With decryption, we can see how HTTP/2 interleaves multiple logical streams for parallel resource loading. This demonstrates HTTP/2's key performance feature that was completely invisible when the traffic was encrypted.

Find concurrent streams

tshark -r http2_decrypted.pcapng -o tls.keylog_file:/tmp/sslkeys.log -Y "http2.type == 0" -T fields -e tcp.stream -e http2.streamid | awk '{print $1}' | sort -u | wc -l

This command quantifies HTTP/2 multiplexing by counting unique TCP streams carrying multiple HTTP/2 streams.

```
student@ubuntu:/tmp$ tshark -r http2_decrypted.pcapng -o tls.keylog_
file:/tmp/sslkeys.log -Y "http2" -T fields -e http2.streamid -e http
2.type -e http2.length | sort -n | head -30
0,0     4,8     18,4
0,0     4,8     18,4
0,0     4,8     18,4
0,0     4,8     18,4
0,0     4,8     18,4
0,0     4,8     18,4
0       4       0
0       4       0
0       4       0
0       4       0
0       4       0
0       4       0
0,0,0,1 4,8,4,1 18,4,0,115
0,0,0,1 4,8,4,1 18,4,0,118
0,0,0,1 4,8,4,1 18,4,0,118
0,0,0,1 4,8,4,1 18,4,0,58
0,0,0,1 4,8,4,1 18,4,0,58
0,0,0,1 4,8,4,1 18,4,0,58
1       0       162
1       0       162
1       0       162
1       0       269
1       0       5911
```

```
student@ubuntu:/tmp$ tshark -r http2_decrypted.pcapng -o tls.keylog_
file:/tmp/sslkeys.log -Y "http2.type == 0" -T fields -e tcp.stream -
e http2.streamid | awk '{print $1}' | sort -u | wc -l
6
```

Step 5.4: Header Compression Analysis

Attempt to view decompressed headers

tshark -r http2_decrypted.pcapng -o tls.keylog_file:/tmp/sslkeys.log -Y "http2.headers" -V | grep -A5 -B5 "Headers:" | head -50

This command reveals HTTP/2 header compression (HPACK) by showing decompressed headers from the decrypted traffic. Unlike HTTP/1.1's plaintext headers, HTTP/2 headers are compressed using HPACK to reduce overhead. With decryption,

we can analyze this efficiency feature that was previously hidden within encrypted TLS packets.

```
student@ubuntu:/tmp$ tshark -r http2_decrypted.pcapng -o tls.keylog_
file:/tmp/sslkeys.log -Y "http2.headers" -V | grep -A5 -B5 "Headers:
" | head -50
        Type: HEADERS (1)
        Flags: 0x05, End Headers, End Stream
            00.0 ..0. = Unused: 0x00
            ..0. .... = Priority: False
            .... 0... = Padded: False
            .... .1.. = End Headers: True
            .... ...1 = End Stream: True
        0... .... .... .... .... .... .... .... = Reserved: 0x0
        .000 0000 0000 0000 0000 0000 0000 0001 = Stream Identifier:
 1
        [Pad Length: 0]
        Header Block Fragment: 828741880be25c2e3cb85717847a8825b650c
3cbb6b83f53032a2f2a
--
        Type: HEADERS (1)
        Flags: 0x04, End Headers
            00.0 ..0. = Unused: 0x00
            ..0. .... = Priority: False
            .... 0... = Padded: False
            .... .1.. = End Headers: True
            .... ...0 = End Stream: False
        0... .... .... .... .... .... .... .... = Reserved: 0x0
        .000 0000 0000 0000 0000 0000 0000 0001 = Stream Identifier:
```

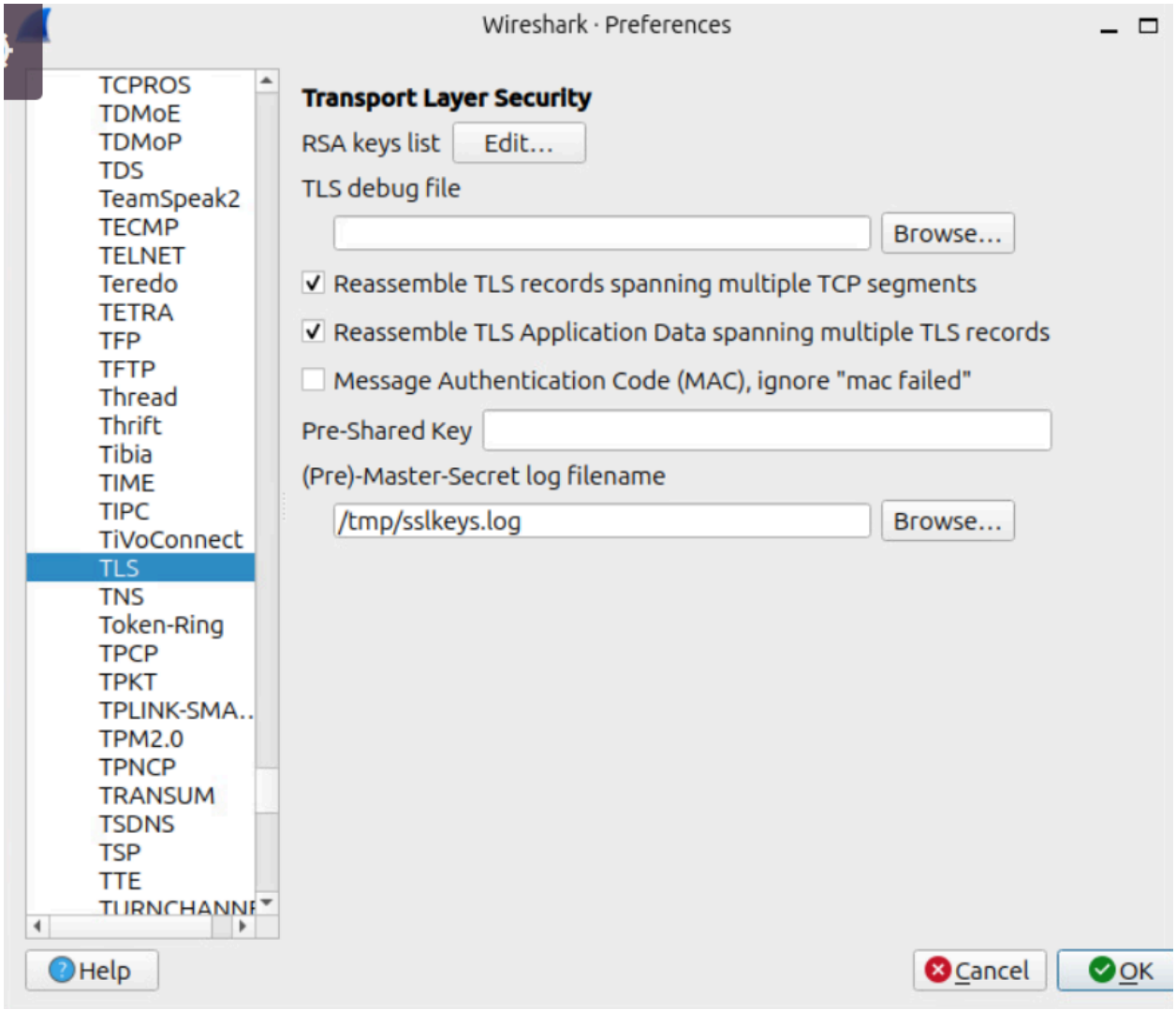
Phase 6: Wireshark Packet Analysis

Step 6.1: Load Wireshark and Perform Decryption

Load Capture in Wireshark

wireshark http2_decrypted.pcapng

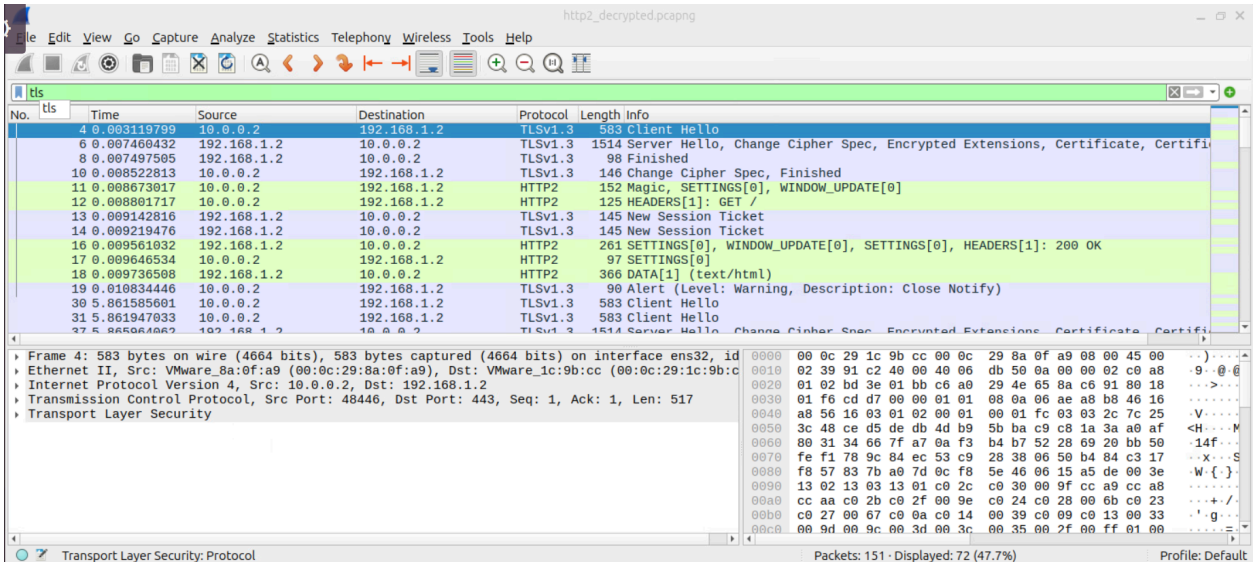
To apply decryption, go to **Edit > Preferences > Protocols > TLS > (Pre)-Master-Secret log filename > Browse… > /tmp/sslkeys.log > OK**



Step 6.2: Analyze TLS Handshake

Apply filter "tls" (without quotation marks). Take a look at the packets. You will see the following:

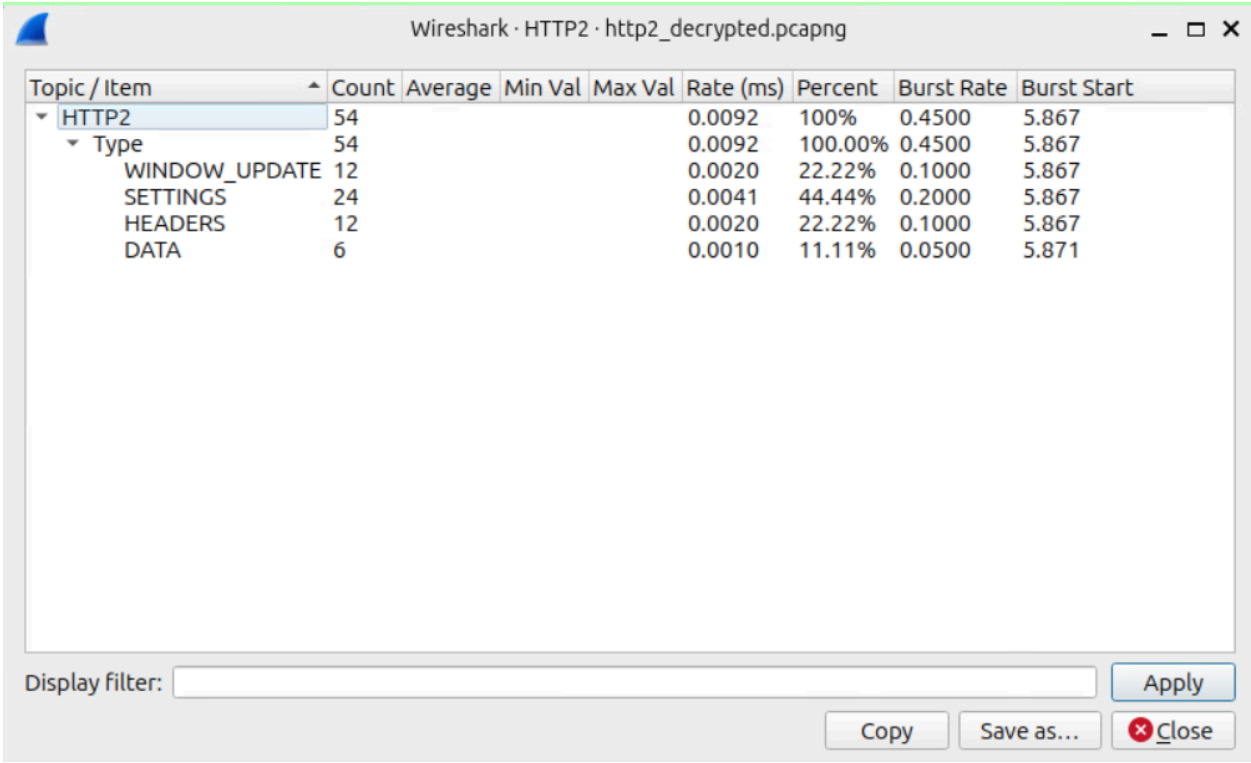
6. Client Hello
7. Server Hello
8. Certificate Exchange
9. Key Exchange
10. Application Data



Step 6.3: Analyze HTTP/2 in Wireshark

Apply filter “http2” (without quotation marks). **Statistics > HTTP/2** and observe:

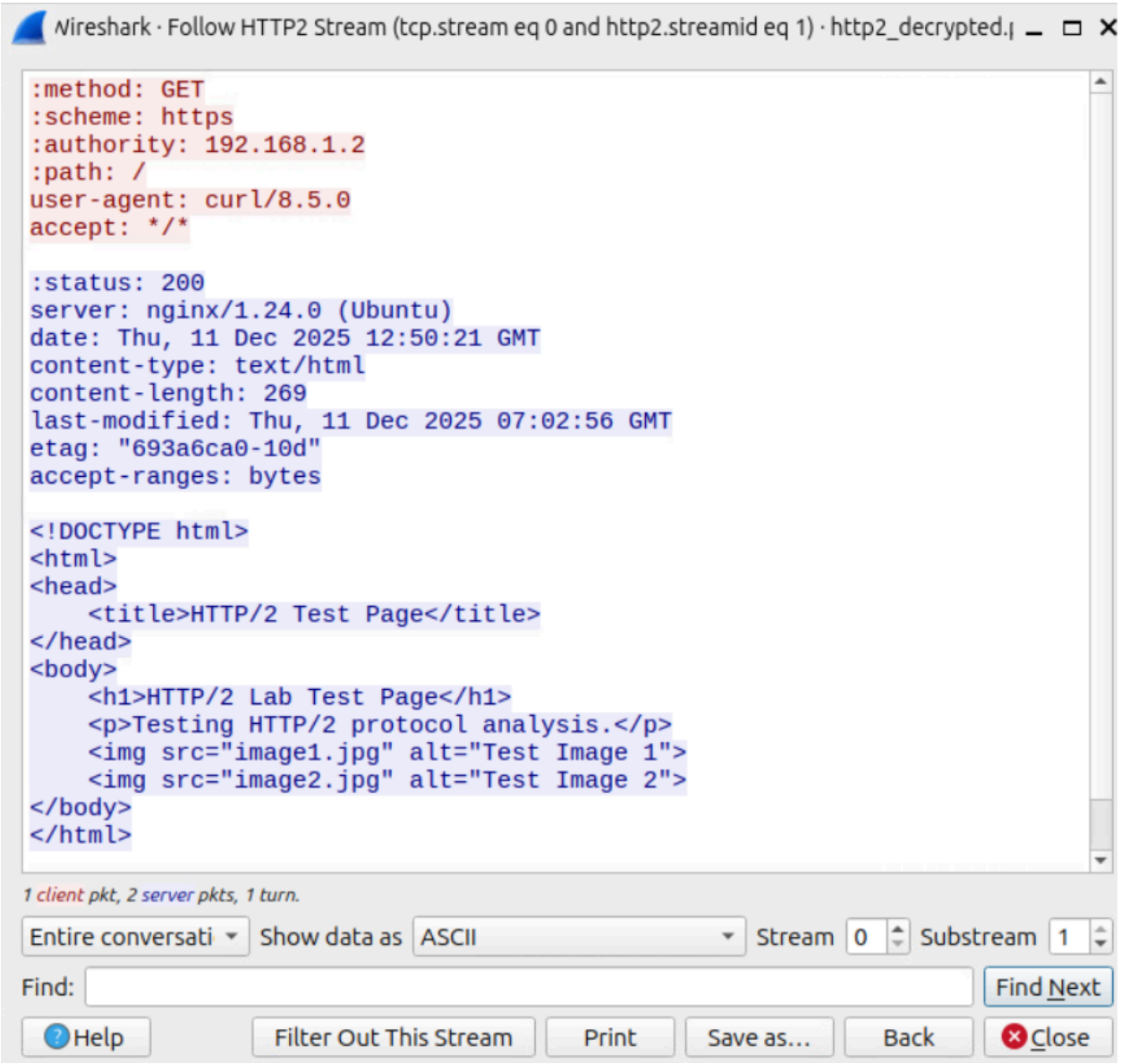
1. Streams graph
2. Frame types distribution
3. Settings exchange



Step 6.4: Follow HTTP/2 Stream

Right-click on HTTP/2 frame with **GET /. Follow > HTTP/2 Stream.** Analyze request/response pattern.

The "Follow HTTP/2 Stream" feature now works because decryption exposes the complete HTTP/2 conversation. This allows Wireshark to reconstruct and display the full request/response pattern for individual streams, showing how HTTP/2 multiplexes multiple logical conversations within a single TCP connection. This detailed stream analysis was impossible before decryption since the HTTP/2 frames were hidden within encrypted TLS Application Data.



Conclusion

This lab covered the setup and forensic analysis of encrypted HTTP/2 traffic. We configured NGINX with TLS 1.3 and HTTP/2, then used SSL key logging to decrypt the traffic for inspection. With tshark and Wireshark, we analyzed HTTP/2 features like multiplexing and header compression. This process showed how encryption forces forensic methods to evolve.

It is important to note that real-world SSL key extraction is very difficult. Legal rules, security policies, and technical barriers like Perfect Forward Secrecy often prevent it.

Therefore, the decryption methods used here are mostly for learning and rarely work outside controlled lab environments.

Key technical lessons were learned. Preparation is critical because keys must be captured before traffic starts. Using both command-line and graphical tools together improves analysis. Also, knowing HTTP/2's structure is essential to understand what the decrypted data shows.

Good forensic methodology involves a layered approach. Start with high-level statistics, then examine frame details. Connecting TLS handshake details to HTTP/2 stream activity helps rebuild the full session. Keeping thorough records with screenshots and outputs supports verification and troubleshooting.

In summary, this lab highlights the balance between encryption for security and the need for forensic visibility. While analysis is possible with the right setup, real investigations usually lack decryption keys. As protocols advance, forensic professionals must adapt their approaches to work within these technical and practical limits.

References

1. **What is HTTP/2?:** https://www.geeksforgeeks.org/computer-networks/what-is-http-2/
2. **Hypertext Transfer Protocol Version 2 (HTTP/2):** https://httpwg.org/specs/rfc7540.html
3. **What is TLS (Transport Layer Security)?:** https://www.cloudflare.com/learning/ssl/transport-layer-security-tls/
4. **What is an SSL/TLS Certificate?:** https://aws.amazon.com/what-is/ssl-certificate/
5. **How To Set Up Nginx with HTTP/2 Support on Ubuntu 20.04:** https://www.digitalocean.com/community/tutorials/how-to-set-up-nginx-with-http-2-support-on-ubuntu-20-04
6. **How to capture and decode HTTP/2 traffic with Wireshark:** https://vanwilgenburg.wordpress.com/2015/11/22/how-to-capture-and-decode-http2-traffic-with-wireshark/
7. **The Ultimate NGINX Installation & Configuration Guide for Ubuntu!:** https://www.youtube.com/watch?v=n7vKxkMIBM0
8. **HOW TO ENABLE HTTP/2 ON NGINX:** https://www.youtube.com/watch?v=O9LX0ej4doY&t=176s